



VIT-AP
UNIVERSITY
Apply Knowledge. Improve Life!®

Final Report

Project Title:-

SkyWrite In The Wind Using Python And OpenCv

Course: Summer Internship Project (22 Batch MTCSE & MTSE)

Presented by: 1) C.Phani Bhushan Reddy [Reg. No. 22MIC7270]

2) V. Jaya Chandra [Reg. No. 22MIC7215]

3) K. Vishwesh Aryan [Reg. No. 22MIC7109]

4) M.Teja Kiran [Reg. No. 22MIC7245]

Project Guide :Prof: Shaik subhani

1. Project Summary

This project is a real-time, touchless drawing application that transforms a standard webcam into a human-computer interaction (HCI) device. Using computer vision, the system tracks the user's hand gestures, allowing them to draw, erase, and create shapes in "thin air." The application features an advanced UI, a variety of tools, and an integrated AI model (TensorFlow) capable of recognizing and replacing hand-drawn digits with clean, typed text.

2. Objectives

The primary goals of this project were to:

- **Create a touchless interface** for digital drawing, removing the need for a mouse or tablet.
- **Implement real-time hand tracking** to follow the user's finger movements.
- **Develop an intuitive gesture control system** (e.g., one finger to draw, two fingers to select).
- **Build a feature-rich UI** with tools like a color palette, shape drawing (rectangle, circle, cube), line thickness control, and an eraser.
- **Integrate an AI model** to provide an "Auto-Write" feature for recognizing hand-drawn numbers.
- **Ensure a smooth user experience** by implementing line-smoothing algorithms to reduce hand jitter.

3. Tools & Technologies Used

- **Python 3:** The core programming language for the project.
- **OpenCV (cv2):** The primary library for all computer vision tasks, including capturing the webcam feed, drawing shapes and lines, creating UI buttons, and displaying the final video windows.
- **Mediapipe (mp):** Google's library used for high-fidelity hand tracking. It provides the 21 3D landmarks of the hand in real-time.
- **TensorFlow (tf.keras):** The AI and machine learning library. We used it to build and run the model that recognizes hand-drawn digits (based on the MNIST dataset).
- **NumPy (np):** A fundamental math library used to create and manage the blank white paintWindow (as a multi-dimensional array of pixels).
- **Pillow (PIL):** An image processing library used to draw the clean, typed text onto the canvas.

after the AI recognizes a number.

- **Collections (deque):** A Python utility used to create the point buffer for our line-smoothing algorithm.
- **OS, Time, Datetime:** Built-in Python libraries used for file checking, button click delays, and creating unique filenames for saved images.

4. Methodology & Implementation

The application functions by processing video one frame at a time in a continuous loop.

1. Setup:

- The webcam is initialized and set to a 1280x720 resolution.
- Mediapipe's Hands solution is loaded.
- A permanent, white paintWindow (a NumPy array) is created to match the webcam's size.
- The Button class and all UI buttons are initialized and placed in a header bar at the top of the screen.

2. Gesture Control (Core Logic):

- For each frame, the hand landmarks are detected.
- We check the position of the Index Finger (Tip 8) and Middle Finger (Tip 12).
- Selection Mode (Cursor) 🖱️: If both the Index and Middle fingers are raised, the user is in "Selection Mode." The tip of the index finger acts as a cursor. If this cursor hovers over a button, it triggers the button's action (e.g., changing colour, changing mode to "RECT").
- Drawing Mode 🖍️: If only the Index finger is raised, the user is in "Drawing Mode."
- Idle Mode: If neither gesture is active, the drawing state is reset.

3. Drawing & Smoothing:

- When in "Drawing Mode," the current (x, y) coordinates of the index finger are captured.
- To prevent shaky lines, these points are added to a deque (a small buffer).
- The average of all points in the deque is calculated.
- A line is drawn on the paintWindow from the *previous* averaged point to the *current* averaged point, creating a smooth, continuous line.
- If the mode is "ERASER," this exact same logic applies, but the color is set to white.

4. UI & Display:

- Two windows are shown:
 - Paint: This window shows *only* the clean white canvas with the permanent drawing.
 - Output: This is the main window. It is a blend. First, the live webcam feed is

drawn. Then, the UI (header bar, buttons) is drawn on top. Finally, this combined image is blended with the Paint window, creating a "ghost" effect of the drawing over the live video.

5. Key Features

- **Multi-Tool Drawing:** Pen, Eraser, Rectangle, Circle, and 2D Cube.
- **Intuitive UI:** A clean header bar contains all tools, colors, and options, leaving the canvas clear.
- **Full Gesture Control:** Two-finger selection and one-finger drawing.
- **Tool Options:** 8-color palette and 5-step brush thickness control.
- **Canvas Control:** A "Clear" button to wipe the canvas and a "Save" button to save the drawing as a PNG file.
- **Line Smoothing:** A point buffer averages the user's hand movements to create clean lines and filter out jitter.
- **AI Auto-Write:** A TensorFlow model that recognizes drawn digits 0-9 and replaces them with typed text.

6. Challenges & Solutions

- **Challenge:** Severe `ModuleNotFoundError` and dependency conflicts, especially between tensorflow, mediapipe, and protobuf.
- **Solution:** We had to forcefully uninstall all conflicting libraries and reinstall them in a specific order, sometimes locking in a specific version (like `tensorflow==2.16`) that was known to be compatible with the others.
- **Challenge:** Hand movements were "shaky," and fast movements created gaps in the lines.
- **Solution:** We implemented a deque (point buffer). By averaging the last 4 frames of hand data, we created a "smoothed" point that filters out small jitters and helps connect lines that were drawn quickly.
- **Challenge:** The Eraser logic was buggy. It would either not work or would conflict with the "Draw" tool.
- **Solution:** We re-coded the button logic. The "ERASER" button now sets the mode to ERASER *and* the colour to WHITE. The "DRAW" button was also made smarter: if the user clicks "DRAW" while the colour is white, it automatically switches the colour to black to prevent "invisible drawing."

7. Future Scope

While the project is fully functional, it could be expanded in several ways:

- **Full Alphabet Recognition:** Expand the AI model to recognize all letters, not just numbers.
- **Undo/Redo:** Implement a stack-based system to store canvas states and allow for undoing actions.
- **Icon-Based UI:** Replace the text on buttons (e.g., "Pen," "Rect") with graphical icons for a more professional look.
- **Additional Shapes:** Add new tools like "Line," "Arrow," or "Triangle."

8. Project Code:

```
9. import cv2
10. import numpy as np
11. import mediapipe as mp
12. from datetime import datetime
13. import tensorflow as tf
14. from tensorflow.keras.models import Sequential, load_model
15. from tensorflow.keras.layers import Dense, Flatten
16. from PIL import Image, ImageDraw, ImageFont
17. import os
18. import time
19. from collections import deque # <-- IMPORT DEQUE FOR SMOOTHING
20.
21. # --- 1. SETUP & CONFIGURATION ---
22. MODEL_PATH = 'my_local_mnist_model.h5'
23. FONT_PATH = "font.ttf"
24. FONT_SIZE = 50
25.
26. # --- 2. HELPER FUNCTIONS & CLASSES ---
27.
28. def get_or_create_model():
29.     """
30.     Loads a pre-trained Keras model from disk. If the model file doesn't exist,
31.     it trains a new simple model on the MNIST dataset and saves it.
32.     """
33.     if os.path.exists(MODEL_PATH):
34.         print("Loading locally saved AI model...")
35.         return load_model(MODEL_PATH)
36.     else:
37.         print("No local model found. Training a new model now (this will take a
38.             minute)...")
39.         try:
40.             (x_train, y_train), _ = tf.keras.datasets.mnist.load_data()
```

```

41.         model = Sequential([
42.             Flatten(input_shape=(28, 28)),
43.             Dense(128, activation='relu'),
44.             Dense(10, activation='softmax')
45.         ])
46.         model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
47.         model.fit(x_train, y_train, epochs=5)
48.         model.save(MODEL_PATH)
49.         print(f"Model trained and saved as {MODEL_PATH}")
50.         return model
51.     except Exception as e:
52.         print(f"Could not train the model. Error: {e}")
53.         return None
54.
55. class Button:
56.     """A class to create and manage clickable UI buttons."""
57.     def __init__(self, x, y, width, height, text, color=(200, 200, 200), icon_path=None):
58.         self.rect = (x, y, width, height)
59.         self.text = text
60.         self.color = color
61.         self.last_click_time = 0
62.         self.icon = None
63.
64.     def draw(self, image, highlight=False):
65.         """Draws the button on a given image."""
66.         x, y, w, h = self.rect
67.         btn_color = (0, 255, 0) if highlight else self.color
68.
69.         overlay = image.copy()
70.         cv2.rectangle(overlay, (x, y), (x + w, y + h), btn_color, -1)
71.         alpha = 0.6 # Transparency factor
72.         image = cv2.addWeighted(overlay, alpha, image, 1 - alpha, 0)
73.         cv2.rectangle(image, (x, y), (x + w, y + h), (50, 50, 50), 2) # Add border
74.
75.         if self.icon is not None:
76.             pass # Icon drawing logic would go here
77.         elif self.text:
78.             text_size = cv2.getTextSize(self.text, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)[0]
79.             text_x = x + (w - text_size[0]) // 2
80.             text_y = y + (h + text_size[1]) // 2
81.             cv2.putText(image, self.text, (text_x, text_y), cv2.FONT_HERSHEY_SIMPLEX,
0.6, (0, 0, 0), 2)
82.
83.         return image # Return the modified image
84.
85.     def is_clicked(self, point):
86.         """Checks if a point is within the button's bounds and adds a debounce delay."""
87.         x, y, w, h = self.rect
88.         current_time_ms = time.time() * 1000
89.         if x < point[0] < x + w and y < point[1] < y + h and (current_time_ms -
self.last_click_time) > 500: # 0.5 sec delay
90.             self.last_click_time = current_time_ms
91.             return True
92.         return False
93.

```

```

94.# --- 3. MAIN APPLICATION CLASS ---
95.
96.class AirCanvas:
97.    """The main class that encapsulates the entire Air Canvas application."""
98.    def __init__(self):
99.        """Initializes the application, loads resources, and sets up the state."""
100.        self.model, self.font = None, None
101.        try:
102.            self.model = get_or_create_model(); self.font =
ImageFont.truetype(FONT_PATH, FONT_SIZE)
103.        except Exception as e: print(f"Error loading resources: {e}")
104.
105.        self.cap = cv2.VideoCapture(0)
106.        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
107.        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
108.
109.        if not self.cap.isOpened(): print("FATAL: Cannot open camera."); return
110.
111.        self.hands = mp.solutions.hands.Hands(max_num_hands=1,
min_detection_confidence=0.7, min_tracking_confidence=0.7)
112.        self.mpDraw = mp.solutions.drawing_utils
113.
114.        self.paint_window = None
115.        self.current_color = (0, 0, 255) # Start with Red
116.        self.current_mode = "DRAW"
117.        self.auto_write_mode = False
118.        self.thickness = 10 # Default thickness
119.        self.max_thickness = 50
120.        self.min_thickness = 2
121.
122.        self.is_drawing = False
123.        self.shape_start_point = None
124.        self.last_point = (0, 0)
125.        self.letter_points = []
126.
127.        # --- *** THE FIX IS HERE *** ---
128.        # A smaller buffer is more responsive and reduces gaps/lag
129.        self.point_buffer = deque(maxlen=4)
130.
131.        # UI Layout
132.        self.header_height = 70
133.        button_size = 50
134.        gap = 10
135.        current_x = gap
136.
137.        self.screen_width = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH))
138.        self.screen_height = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
139.
140.        self.buttons = {
141.            "DRAW": Button(current_x, gap, button_size, button_size, "Pen"),
142.            "RECT": Button(current_x + (button_size+gap), gap, button_size,
button_size, "Rect"),
143.            "CIRCLE": Button(current_x + 2*(button_size+gap), gap, button_size,
button_size, "Circ"),
144.            "CUBE": Button(current_x + 3*(button_size+gap), gap, button_size,
button_size, "Cube"),

```

```

145.         "FILL": Button(current_x + 4*(button_size+gap), gap, button_size,
            button_size, "Fill"),
146.         "ERASER": Button(current_x + 5*(button_size+gap), gap, button_size,
            button_size, "Erase"),
147.     }
148.     current_x += 6*(button_size+gap) + 20
149.
150.     self.buttons["THICK+"] = Button(current_x, gap, button_size, button_size, "+")
151.     self.buttons["THICK-"] = Button(current_x + (button_size+gap), gap,
            button_size, button_size, "-")
152.     self.thickness_text_x = current_x
153.     current_x += 2*(button_size+gap) + 20
154.
155.     self.colors_list = [
156.         ("RED", (0,0,255)), ("ORANGE", (0,165,255)), ("YELLOW", (0,255,255)),
157.         ("GREEN", (0,255,0)), ("BLUE", (255,0,0)), ("PURPLE", (128,0,128)),
158.         ("BLACK", (0,0,0)), ("WHITE", (255,255,255)),
159.     ]
160.     color_button_size = 40
161.     for i, (name, bgr) in enumerate(self.colors_list):
162.         self.buttons[name] = Button(current_x, gap + (button_size-
            color_button_size)//2, color_button_size, color_button_size, "", color=bgr)
163.         current_x += (color_button_size + gap)
164.
165.     options_x = self.screen_width - (3 * (button_size + gap))
166.     self.buttons["AUTO-W"] = Button(options_x, gap, button_size, button_size, "AI")
167.     self.buttons["CLEAR"] = Button(options_x + (button_size+gap), gap, button_size,
            button_size, "Clear")
168.     self.buttons["SAVE"] = Button(options_x + 2*(button_size+gap), gap,
            button_size, button_size, "Save")
169.
170.     def run(self):
171.         """The main loop of the application."""
172.         while self.cap.isOpened():
173.             ret, frame = self.cap.read()
174.             if not ret: print("Error: Could not read frame..."); break
175.             frame = cv2.flip(frame, 1)
176.             if self.paint_window is None: self.paint_window = np.full(frame.shape, 255,
                dtype=np.uint8)
177.
178.             framergb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
179.             result = self.hands.process(framergb)
180.
181.             output_frame = frame.copy()
182.
183.             cv2.rectangle(output_frame, (0, 0), (self.screen_width,
                self.header_height), (230, 230, 230), -1)
184.
185.             if result.multi_hand_landmarks:
186.                 hand_landmarks = result.multi_hand_landmarks[0]
187.                 self.handle_gestures(hand_landmarks, output_frame)
188.                 self.mpDraw.draw_landmarks(output_frame, hand_landmarks,
                    mp.solutions.hands.HAND_CONNECTIONS)
189.
190.             output_frame = self.draw_all_ui(output_frame)

```



```

191.
192.         cv2.putText(output_frame, f"Thick:{self.thickness}", (self.thickness_text_x
+ 2*(self.buttons["THICK+"].rect[2]+10), 40), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,0), 2)
193.
194.         final_frame = cv2.addWeighted(output_frame, 0.7, self.paint_window, 0.3, 0)
195.
196.         cv2.imshow("Output", final_frame)
197.         cv2.imshow("Paint", self.paint_window)
198.
199.         if cv2.waitKey(1) == ord('q'): break
200.
201.     self.cap.release(); cv2.destroyAllWindows()
202.
203.     def handle_gestures(self, hand_landmarks_proto, output_frame):
204.         """The main logic to interpret gestures and control the application."""
205.         landmarks = [(int(lm.x * self.screen_width), int(lm.y * self.screen_height))
for lm in hand_landmarks_proto.landmark]
206.
207.         index_tip = landmarks[8]
208.         middle_tip = landmarks[12]
209.
210.         is_index_up = index_tip[1] < landmarks[6][1]
211.         is_middle_up = middle_tip[1] < landmarks[10][1]
212.
213.         # GESTURE 1: SELECTION MODE (Index + Middle are up)
214.         if is_index_up and is_middle_up:
215.             cursor_pos = index_tip
216.             if self.is_drawing:
217.                 self.finalize_action()
218.
219.             self.is_drawing = False
220.             self.last_point = (0,0)
221.             self.point_buffer.clear() # Clear smoothing buffer
222.
223.             cv2.circle(output_frame, cursor_pos, 10, (0, 255, 0), 2)
224.
225.             for name, button in self.buttons.items():
226.                 if button.is_clicked(cursor_pos):
227.                     self.handle_button_click(name)
228.
229.         # GESTURE 2: DRAWING MODE (Only Index is up)
230.         elif is_index_up and not is_middle_up:
231.             cursor_pos = index_tip
232.             if not self.is_drawing:
233.                 self.is_drawing = True
234.                 self.shape_start_point = cursor_pos
235.                 self.letter_points = []
236.                 self.point_buffer.clear() # Clear buffer
237.                 self.last_point = cursor_pos # Set first point
238.
239.             cv2.circle(output_frame, cursor_pos, 10, (0, 0, 255), 2)
240.
241.             self.perform_action(cursor_pos, output_frame)
242.             # self.last_point is now updated inside perform_action
243.
244.         # GESTURE 3: IDLE (Fingers down)

```

```

245.         else:
246.             if self.is_drawing:
247.                 self.finalize_action()
248.                 self.is_drawing = False
249.                 self.last_point = (0,0)
250.                 self.point_buffer.clear() # Clear smoothing buffer
251.
252.     def perform_action(self, finger_tip, output_frame):
253.         """Executes the current drawing/preview action."""
254.
255.         # --- SMOOTHING LOGIC ---
256.         self.point_buffer.append(finger_tip)
257.         if len(self.point_buffer) > 1:
258.             avg_x = sum(p[0] for p in self.point_buffer) // len(self.point_buffer)
259.             avg_y = sum(p[1] for p in self.point_buffer) // len(self.point_buffer)
260.             draw_point = (avg_x, avg_y)
261.         else:
262.             draw_point = finger_tip
263.         # --- END OF SMOOTHING LOGIC ---
264.
265.         color = (255,255,255) if self.current_mode == "ERASER" else self.current_color
266.
267.         if self.current_mode == "DRAW" or self.current_mode == "ERASER":
268.             if self.last_point != (0,0):
269.                 cv2.line(self.paint_window, self.last_point, draw_point, color,
270. self.thickness)
271.
272.                 if self.auto_write_mode and self.current_mode == "DRAW":
273.                     self.letter_points.append(draw_point)
274.
275.                 self.last_point = draw_point
276.
277.         if self.current_mode in ["RECT", "CIRCLE", "CUBE"]:
278.             self.draw_shape_preview(self.shape_start_point, finger_tip, output_frame)
279.             # Update last_point for shape finalization
280.             self.last_point = finger_tip
281.
282.     def finalize_action(self):
283.         """Draws the final shape/action onto the permanent canvas."""
284.         # Use self.last_point (which was the raw finger tip position) for shapes
285.         raw_last_point = self.last_point
286.
287.         if self.current_mode == "RECT": cv2.rectangle(self.paint_window,
288. self.shape_start_point, raw_last_point, self.current_color, -1)
289.         if self.current_mode == "CIRCLE":
290.             radius = int(np.linalg.norm(np.array(self.shape_start_point) -
291. np.array(raw_last_point)))
292.             cv2.circle(self.paint_window, self.shape_start_point, radius,
293. self.current_color, -1)
294.         if self.current_mode == "CUBE": self.draw_cube(self.shape_start_point,
295. raw_last_point, self.paint_window)
296.         if self.current_mode == "FILL": cv2.floodFill(self.paint_window, None,
297. seedPoint=raw_last_point, newVal=self.current_color)
298.         if self.auto_write_mode and self.current_mode == "DRAW" and
299. len(self.letter_points) > 10:

```

```

293.         self.perform_ocr()
294.
295.     def handle_button_click(self, name):
296.         """Handles the logic for when a UI button is clicked."""
297.
298.         if name in ["RECT", "CIRCLE", "CUBE", "FILL"]:
299.             self.current_mode = name
300.
301.         elif name == "DRAW":
302.             self.current_mode = "DRAW"
303.             if self.current_color == (255, 255, 255):
304.                 self.current_color = (0, 0, 0) # Default to black
305.
306.         elif name == "ERASER":
307.             self.current_mode = "ERASER"
308.             self.current_color = (255, 255, 255) # Set color to white
309.
310.         elif name == "AUTO-W":
311.             self.auto_write_mode = not self.auto_write_mode
312.         elif name == "CLEAR":
313.             self.paint_window.fill(255)
314.         elif name == "SAVE":
315.             cv2.imwrite(f"drawing_{datetime.now().strftime('%Y%m%d_%H%M%S')}.png",
self.paint_window)
316.         elif name == "THICK+":
317.             self.thickness = min(self.max_thickness, self.thickness + 5)
318.         elif name == "THICK-":
319.             self.thickness = max(self.min_thickness, self.thickness - 5)
320.
321.         elif name in [c[0] for c in self.colors_list]: # Color button clicked
322.             self.current_color = self.buttons[name].color
323.             if name == "WHITE":
324.                 self.current_mode = "ERASER"
325.             else:
326.                 self.current_mode = "DRAW"
327.
328.     def draw_all_ui(self, image):
329.         """Draws all UI buttons on the screen."""
330.         for name, button in self.buttons.items():
331.             highlight = (name == self.current_mode) or \
332.                 (name == "AUTO-W" and self.auto_write_mode) or \
333.                 (hasattr(button, 'color') and button.color ==
self.current_color and name not in ["DRAW", "RECT", "CIRCLE", "CUBE", "FILL", "ERASER",
"AUTO-W", "CLEAR", "SAVE", "THICK+", "THICK-"])
334.
335.             image = button.draw(image, highlight)
336.         return image # Return the final image with all buttons drawn
337.
338.     def perform_ocr(self):
339.         """Performs character recognition on the last drawn stroke."""
340.         if not self.letter_points: return
341.         min_x=max(0,min(p[0] for p in self.letter_points)-20);
max_x=min(self.paint_window.shape[1],max(p[0] for p in self.letter_points)+20)
342.         min_y=max(0,min(p[1] for p in self.letter_points)-20);
max_y=min(self.paint_window.shape[0],max(p[1] for p in self.letter_points)+20)

```

```

343.         letter_roi=self.paint_window[min_y:max_y,min_x:max_x];
344.         cv2.rectangle(self.paint_window,(min_x,min_y),(max_x,max_y),(255,255,255),-1)
345.         if letter_roi.size > 0:
346.             recognized_char=self.recognize_character(letter_roi)
347.             if recognized_char and self.font:
348.                 pil_img=Image.fromarray(self.paint_window);
349.                 draw=ImageDraw.Draw(pil_img)
350.                 draw.text((min_x+5,min_y-
351.                 10),recognized_char,font=self.font,fill=tuple(reversed(self.current_color)));
352.                 self.paint_window=np.array(pil_img)
353.
354.     def recognize_character(self, img):
355.         """Pre-processes an image and uses the AI model to predict the character."""
356.         if self.model is None: return None
357.         img_gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY);
358.         img_inverted=cv2.bitwise_not(img_gray)
359.         kernel=np.ones((5,5),np.uint8);
360.         img_dilated=cv2.dilate(img_inverted,kernel,iterations=2)
361.         img_resized=cv2.resize(img_dilated,(28,28),interpolation=cv2.INTER_AREA);
362.         img_resized=img_resized.reshape(1,28,28).astype('float32')/255.0
363.         prediction=self.model.predict(img_resized); confidence=np.max(prediction)
364.         if confidence>0.8: return str(np.argmax(prediction))
365.         return None
366.
367.     def draw_shape_preview(self, p1, p2, image):
368.         """Draws a live preview of a shape on the output frame."""
369.         color = self.current_color
370.         if self.current_mode=="RECT": cv2.rectangle(image, p1, p2, color, 2)
371.         if self.current_mode=="CIRCLE": radius=int(np.linalg.norm(np.array(p1)-
372.         np.array(p2))); cv2.circle(image, p1, radius, color, 2)
373.         if self.current_mode=="CUBE": self.draw_cube(p1, p2, image, is_preview=True)
374.
375.     def draw_cube(self, p1, p2, image, is_preview=False):
376.         """Draws a 2D representation of a cube."""
377.         thickness = 2
378.         color = self.current_color
379.         cv2.rectangle(image, p1, p2, color, thickness)
380.         offset = int(np.linalg.norm(np.array(p1)-np.array(p2)) * 0.4)
381.         p3, p4 = (p1[0] + offset, p1[1] - offset), (p2[0] + offset, p2[1] - offset)
382.         cv2.rectangle(image, p3, p4, color, thickness)
383.         cv2.line(image, p1, p3, color, thickness); cv2.line(image, (p2[0], p1[1]),
384.         (p4[0], p3[1]), color, thickness)
385.         cv2.line(image, (p1[0], p2[1]), (p3[0], p4[1]), color, thickness);
386.         cv2.line(image, p2, p4, color, thickness)
387.
388. # --- 4. RUN THE APPLICATION ---
389. if __name__ == "__main__":
390.     app = AirCanvas()
391.     app.run()

```

THANKYOU.

